COMPILER DESIGN

Semester	VI				
Course code					
Category	Professional Core Courses				
Course title	Compiler Design				
Scheme and Credits	L	T	P	Credits	
	3	0	0	3	
Classwork	30 Marks				
Exam	70 Marks				
Total	100 Marks				
Duration of Exam	03 Hours				

Note: The examiner will set nine questions in total. Question one will be compulsory. Question one will have seven parts of 2 marks each from all units, and the remaining eight questions of 14 marks each to be set by taking two questions from each unit. The students have to attempt five questions in total, the first being compulsory and selecting one from each unit.

COURSE OBJECTIVE:

1. To understand and list the different stages in the process of compilation.
2. Identify different methods of lexical analysis.
3. Design top-down and bottom-up parsers.
4. Identify synthesized and inherited attributes.
5. Develop syntax-directed translation schemes.

UNIT - I

Introduction to Compilers: Language Processors, The Structure of compiler: its different phases, Compiler Construction Tools, Applications of Compiler Technology.

Lexical Analysis: Role of lexical analyzer, Input Buffering, Specification, and recognition of tokens, design of lexical analyzer, regular expressions, A language specifying lexical analyzer, Finite automata, conversion from regular expression to finite automata, and vice versa, minimizing the number of states of DFA, Implementation of lexical analyzer.

UNIT - II

Syntax Analysis: Role of parsers, context-free grammars.

Parsing Technique: Shift-reduce parsing, Operator precedence parsing, Top-down parsing, Predictive parsing.

UNIT - III

LR parsers, SLR, LALR, and Canonical LR parser.

Syntax Directed Translations: Syntax-directed definitions, construction of syntax trees, syntax-directed translation scheme, implementation of syntax-directed translation, Intermediate-Code Generation: three address code, quadruples and triples.

UNIT - IV

Symbol Table & Error Detection, and Recovery: Symbol tables: its contents and data structure for symbol tables; trees, arrays, linked lists, hash tables. Errors, lexical phase error, syntactic phase

error, and Semantic error. **Code Optimization & Code Generation:** Code generation, forms of objects code, machine-dependent code, optimization, register allocation for temporary and user defined variables.

COURSE OUTCOMES:

At the end of this course, students will demonstrate the ability to

CO1: state principles of compiler design.

CO2: illustrate the essential phases for automatically converting source code into object code.

CO3: apply lexical analysis, syntax analysis and code optimization techniques for solving problems.

CO4: analyse a parse tree and a given BNF grammar.

CO5: compare and contrast syntax-oriented translation schemes.

CO6: design a lexical analyser from the specification of a language's lexical rules.

TEXT AND REFERENCE BOOKS:

1. Theory and practice of compiler writing, Tremblay & Sorenson, 1985, Mc. Graw Hill.
2. System software by Dhamdhere, 1986, MGH.
3. Principles of compiler Design, Narosa Publication
4. Elements compiler Design, Dr. M. Joseph, University Science Press
5. Compilers Principle, Techniques & Tools – Alfred V. AHO, Ravi Sethi & J.D. Ullman; 1998 Addison Wesley.